

Metodos_Numericos_Libro

Julio Trujillo

2026-01-25

Table of contents

1	Índice del libro	3
1.0.1	Comenzar	3
1.0.2	Capítulos	3
2	Introduction	4
3	Summary	5
4	Capítulo 1. Error en el Cálculo Numérico	6
5	Introducción	7
6	Valores reales: exactitud y precisión	8
6.1	Valores exactos y valores aproximados	8
6.2	Exactitud	8
6.3	Precisión	9
6.4	Error relativo	9
7	Almacenamiento interno de los números	10
7.1	Representación binaria	10
7.2	Números de punto flotante	10
8	Error numérico	11
8.1	Error de redondeo	11
8.2	Tipos de redondeo	11
8.3	Pérdida de cifras significativas	11
9	Notación O de Landau	13
10	Propagación del error	14
10.1	Idea general	14
10.2	Propagación en operaciones básicas	14
10.3	Estabilidad numérica	14
11	Conclusiones	15
	References	16

1 Índice del libro

1.0.1 Comenzar

- [Introducción](#)
- [Resumen](#)

1.0.2 Capítulos

- [Capítulo 1](#)

2 Introduction

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

```
1 + 1
```

```
[1] 2
```

3 Summary

In summary, this book has no content whatsoever.

1 + 1

[1] 2

4 Capítulo 1. Error en el Cálculo Numérico

5 Introducción

En el análisis matemático tradicional se trabaja, en muchos casos, con valores exactos y expresiones analíticas cerradas. Sin embargo, en la práctica científica, ingenieril y computacional, los datos disponibles suelen provenir de **mediciones, aproximaciones o cálculos realizados mediante computadoras**, lo que introduce inevitablemente errores.

El **análisis numérico** se ocupa del estudio de algoritmos para obtener soluciones aproximadas a problemas matemáticos, así como del análisis del **error** asociado a dichas aproximaciones. Incluso cuando un método es matemáticamente correcto, su implementación computacional puede producir resultados poco fiables si no se controla adecuadamente la propagación del error.



Ejemplo

Encontrar la raíz de 2 para cuatro cifras decimales.

Usamos cuatro operaciones básicas de la aritmética.

$$x_1 = 1 \quad x_{n+1} = \frac{1}{2} \left(x_n + \frac{2}{x_n} \right)$$

$$\text{Haciendo unos cálculos rápidos tenemos } x_2 = \frac{3}{2} \quad x_3 = \frac{17}{12} \quad x_4 = \frac{1}{2} \left(\frac{17}{12} + \frac{24}{17} \right)$$

$$\text{O redondeando a cuatro decimales } x_2 = 1.5000 \quad x_3 = 1.4167 \quad x_4 = 1.4142$$

En este capítulo se introducen los conceptos fundamentales relacionados con: - la representación de los números reales, - la exactitud y la precisión, - el almacenamiento interno de los números en la computadora, - los distintos tipos de error numérico, - la pérdida de cifras significativas, - la notación de orden de error (notación de Landau), - y la propagación del error.

Estos conceptos serán esenciales para comprender y evaluar los métodos numéricos estudiados en capítulos posteriores.

6 Valores reales: exactitud y precisión

6.1 Valores exactos y valores aproximados

Un **valor exacto** es aquel que representa una cantidad sin error alguno. Ejemplos de valores exactos son: - números enteros definidos, - fracciones racionales expresadas de forma exacta, - constantes matemáticas definidas simbólicamente, como π o $\sqrt{2}$.

Un **valor aproximado** es una representación numérica limitada de un valor real verdadero. En computación, prácticamente todos los números reales se manejan como valores aproximados debido a las restricciones de almacenamiento y cálculo.

En general, si x representa el valor verdadero y $\tilde{x}$ su aproximación, se tiene: $\tilde{x} \approx x$

6.2 Exactitud

La **exactitud** mide qué tan cerca está un valor aproximado del valor real verdadero.

El **error absoluto** se define como: $E_a = \|x - \tilde{x}\|$

y cuantifica la desviación total entre el valor real y su aproximación.



Ejemplo

Sea $x = 3.14159265$ el valor verdadero de π y $\tilde{x} = 3.14$ una aproximación.

El error absoluto es: $E_a = \|3.14159265 - 3.14\| = 0.00159265$

6.3 Precisión

La **precisión** indica el número de cifras significativas con las que se expresa un número, independientemente de su cercanía al valor real.

Por ejemplo: - 3.14 tiene tres cifras significativas, - 3.141592 tiene siete cifras significativas.

Es importante destacar que **exactitud y precisión no son conceptos equivalentes**: - un valor puede ser muy preciso pero poco exacto, - o poco preciso pero razonablemente exacto.

6.4 Error relativo

El **error relativo** se define como: $E_r = \frac{|x - \tilde{x}|}{|x|}$

Este tipo de error es especialmente útil cuando se comparan errores en cantidades de distinta magnitud.



Ejemplo

Si $x = 1000$ y $\tilde{x} = 999$, entonces: $E_a = 1$, $E_r = \frac{1}{1000} = 0.001$

7 Almacenamiento interno de los números

7.1 Representación binaria

Las computadoras representan la información internamente en **sistema binario**, es decir, en base 2.

Un número entero positivo se expresa como: $(d_n d_{n-1} \dots d_0)_2 = \sum_{i=0}^n d_i 2^i$, $d_i \in 0, 1$



Ejemplo

El número decimal 13 se representa en binario como: $13_{10} = 1101_2$

Muchos números decimales no poseen una representación binaria finita, lo que da lugar a errores de aproximación.

7.2 Números de punto flotante

Los números reales se almacenan usualmente en formato de **punto flotante**, compuesto por: - un signo, - una mantisa, - un exponente.

La forma general es: $x = \pm(0.m_1 m_2 \dots m_t)_2 \times 2^e$

Este formato permite representar números muy grandes y muy pequeños, pero introduce **errores de redondeo** inevitables.



Ejemplo clásico

El número decimal 0.1 no tiene representación binaria exacta, ya que: $0.1_{10} = 0.0001100110011 \dots_2$

8 Error numérico

8.1 Error de redondeo

El **error de redondeo** aparece cuando un número real se aproxima usando un número finito de cifras.

Si se redondea un número a n cifras decimales, el error máximo es del orden de: $\frac{1}{2} \times 10^{-n}$



Ejemplo

Al redondear 1.236 a dos decimales se obtiene 1.24, con un error máximo de 0.005.

8.2 Tipos de redondeo

Los principales métodos de redondeo son: - redondeo hacia abajo, - redondeo hacia arriba, - redondeo al número par más cercano (utilizado para reducir sesgos acumulativos).

8.3 Pérdida de cifras significativas

La **pérdida de cifras significativas** (o cancelación) ocurre cuando se restan números muy cercanos entre sí.



Ejemplo

Considérese: $x = 1.0000001$, $y = 1.0000000$ Entonces: $x - y = 0.0000001$

Si los números se almacenan con pocas cifras significativas, el resultado puede perder completamente su precisión.

Este fenómeno es una fuente importante de inestabilidad numérica.

9 Notación O de Landau

La **notación de Landau** se utiliza para describir el orden de magnitud de un error.

Se dice que: $f(h) = O(h^n)$ si $\lim_{h \rightarrow 0} \frac{f(h)}{h^n} = C$

donde C es una constante finita.

Esta notación permite comparar la eficiencia y precisión de distintos métodos numéricos.



Ejemplo

Un método con error $O(h^2)$ es, para h pequeño, más preciso que uno con error $O(h)$.

10 Propagación del error

10.1 Idea general

Cuando un valor aproximado se utiliza como entrada en un cálculo posterior, su error se propaga.

Si: $y = f(x)$, $x = \tilde{x} + \delta x$

entonces, para errores pequeños: $\delta y \approx f'(x)\delta x$

10.2 Propagación en operaciones básicas

- **Suma y resta:** se suman los errores absolutos.
- **Producto y cociente:** se suman los errores relativos.



Ejemplo

Si: $x = 10 \pm 0.1$, $y = 5 \pm 0.05$

entonces: $xy = 50 \pm 50 \left(\frac{0.1}{10} + \frac{0.05}{5} \right) = 50 \pm 1$

10.3 Estabilidad numérica

Un algoritmo es **numéricamente estable** si los errores no se amplifican de manera significativa durante el proceso de cálculo.

Dos algoritmos matemáticamente equivalentes pueden presentar comportamientos numéricos muy distintos al implementarse en una computadora.

11 Conclusiones

El estudio del error es un componente esencial del análisis numérico. Comprender su origen, su comportamiento y su propagación permite: - evaluar la confiabilidad de los resultados computacionales, - comparar métodos numéricos, - diseñar algoritmos estables y eficientes.

References

Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.